

DML

(집합 연산자)

• 집합 연산자(SET OPERATOR)

하나의 쿼리문 안에 여러개의 쿼리가 들어가는 형태를 가지는 연산자로서
테이블의 데이터를 하나로 합칠 때 주로 사용됨

* 참고 : 데이터가 조회된 결과물을 RESULT SET 이라고 부름

종류	설명	기타
UNION	두 개의 쿼리문에 대한 수행 결과를 더하고 중복 제거	합집합
UNION ALL	두 개의 쿼리문에 대한 수행 결과를 더하고 중복 제거하지 않음	합집합+교집합
INTERSECT	두 개의 쿼리문에 대한 수행 결과의 값 중 중복된 부분만 조회	교집합
MINUS	먼저 실행된 첫번째 쿼리문의 수행 결과에서 그 다음 수행된 쿼리문의 수행 결과값을 뺀 나머지 부분	차집합

- UNION

EMPLOYEES 테이블에서 DEPARTMENT_ID가 80이면서 SALARY가 10000 이상인 직원 조회

```
107 SELECT EMPLOYEE_ID, FIRST_NAME, SALARY, DEPARTMENT_ID FROM EMPLOYEES e
108 WHERE DEPARTMENT_ID = 80
109 UNION
110 SELECT EMPLOYEE_ID, FIRST_NAME, SALARY, DEPARTMENT_ID FROM EMPLOYEES e
111 WHERE SALARY >= 10000;
112
```

Results 1

SELECT EMPLOYEE_ID, FIRST_NAME, SALARY, DEPARTMENT_ID

	EMPLOYEE_ID	FIRST_NAME	SALARY	DEPARTMENT_ID
1	145	John	14,000	80
2	146	Karen	13,500	80
3	147	Alberto	12,000	80
4	148	Gerald	11,000	80
5	149	Eleni	10,500	80
6	150	Peter	10,000	80
7	151	David	9,500	80
8	152	Peter	9,000	80

- UNION

EMPLOYEES 테이블에서 급여가 5000이상 10000 이하거나 이름이 A로 시작하는 직원 조회

```
116 SELECT FIRST_NAME, LAST_NAME, SALARY
117 FROM EMPLOYEES
118 WHERE SALARY BETWEEN 5000 AND 10000
119 UNION
120 SELECT FIRST_NAME, LAST_NAME, SALARY
121 FROM EMPLOYEES
122 WHERE FIRST_NAME LIKE 'A%';
```

Results 1

SELECT FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES Enter a SQL expression to filter results (use Ctrl+Space)

	FIRST_NAME	LAST_NAME	SALARY
1	Pat	Fay	6,000
2	Susan	Mavris	6,500
3	Hermann	Baer	10,000
4	William	Gietz	8,300
5	Alexander	Hunold	9,000

• UNION

아래와 같이 서로 다른 테이블에 대한 결과를 합쳐서 조회할 수 있음

```
125 SELECT DEPARTMENT_ID FROM EMPLOYEES  
126 UNION  
127 SELECT DEPARTMENT_ID FROM DEPARTMENTS;  
128
```

results 1	
SELECT DEPARTMENT_ID FROM EMPLOYEES UNION SELECT DEPARTMENT_ID FROM DEPARTMENTS	
DEPARTMENT_ID	
50	
10	
20	
40	
70	
110	
90	
60	
100	
30	

- UNION

EMPLOYEES 테이블과 DEPARTMENTS 테이블의 데이터 결과를 합치고 중복 제거

```
125 SELECT FIRST_NAME, LAST_NAME, MANAGER_ID
126 FROM EMPLOYEES
127 UNION
128 SELECT DEPARTMENT_NAME, to_char(LOCATION_ID), MANAGER_ID
129 FROM DEPARTMENTS;
130
131
```

Results 1

SELECT FIRST_NAME, LAST_NAME, MANAGER_ID

	FIRST_NAME	LAST_NAME	MANAGER_ID
114	Public Relations	2700	204
115	Sales	2500	145
116	Executive	1700	100
117	Finance	1700	108
118	Accounting	1700	205
119	Treasury	1700	[NULL]
120	Corporate Tax	1700	[NULL]
121	Control And Cred	1700	[NULL]

• 설명

1. 데이터 조회에 대한 컬럼명은 선행 쿼리문에 의해 결정됨
2. DEPARTMENT_NAME은 FIRST_NAME 컬럼의 결과값으로 더해짐
3. TO_CHAR를 사용한 이유 : LOCATION_ID가 LAST_NAME 컬럼이 있는 공간으로 데이터를 합쳐야하기 때문에 데이터 타입을 맞춰줘야 함
4. MANAGER_ID를 선행 쿼리문의 MANAGER_ID의 결과값에 더함

• UNION ALL

UNION과 동작은 똑같지만 중복을 제거하지 않음

```
116 SELECT FIRST_NAME, LAST_NAME, SALARY
117 FROM EMPLOYEES
118 WHERE SALARY BETWEEN 5000 AND 10000
119 UNION ALL
120 SELECT FIRST_NAME, LAST_NAME, SALARY
121 FROM EMPLOYEES
122 WHERE FIRST_NAME LIKE 'A%';
```

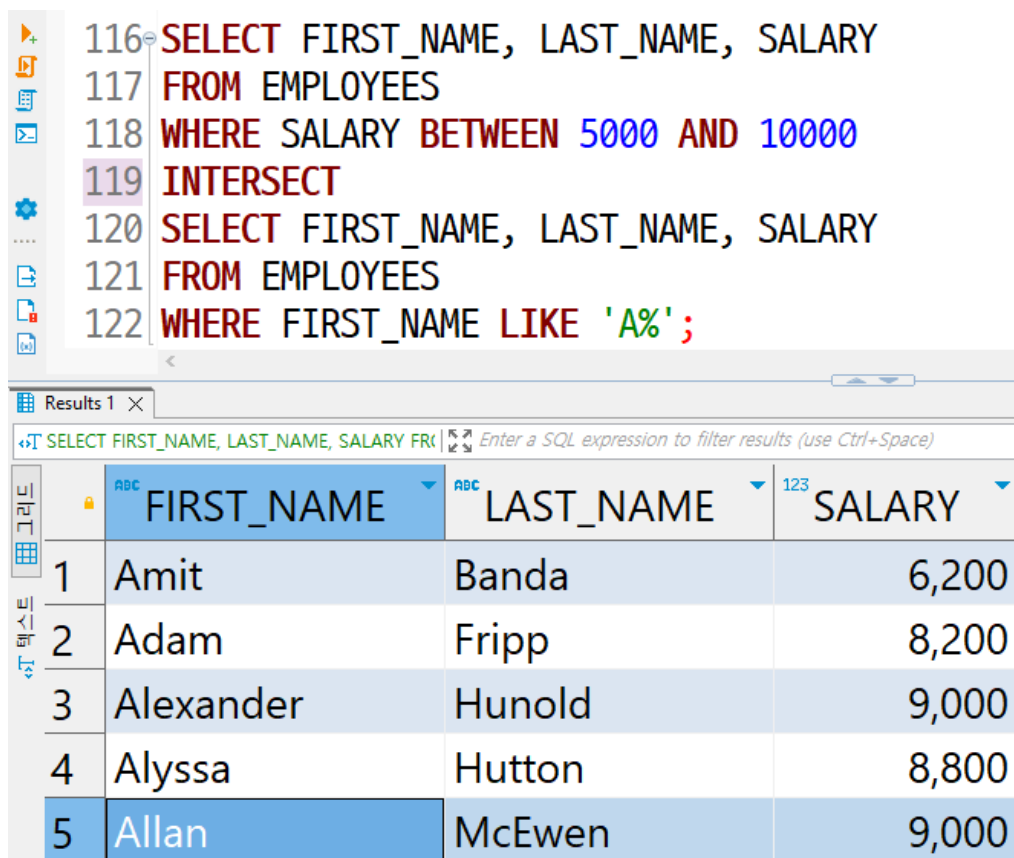
Results 1

SELECT FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES WHERE SALARY BETWEEN 5000 AND 10000 UNION ALL SELECT FIRST_NAME, LAST_NAME, SALARY FROM EMPLOYEES WHERE FIRST_NAME LIKE 'A%';

	FIRST_NAME	LAST_NAME	SALARY
1	Adam	Fripp	8,200
2	Adam	Fripp	8,200
3	Alana	Walsh	3,100
4	Alberto	Errazuriz	12,000
5	Alexander	Hunold	9,000
6	Alexander	Hunold	9,000
7	Alexander	Khoo	3,100
8	Alexis	Bull	4,100
9	Allan	McEwen	9,000
10	Allan	McEwen	9,000

• INTERSECT

두 개의 쿼리문에서 결과값만을 더하여 데이터를 조회함



The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is as follows:

```
116 SELECT FIRST_NAME, LAST_NAME, SALARY
117 FROM EMPLOYEES
118 WHERE SALARY BETWEEN 5000 AND 10000
119 INTERSECT
120 SELECT FIRST_NAME, LAST_NAME, SALARY
121 FROM EMPLOYEES
122 WHERE FIRST_NAME LIKE 'A%';
```

The results pane, titled "Results 1", displays the output of the query in a table format. The table has three columns: FIRST_NAME, LAST_NAME, and SALARY. The results are as follows:

	FIRST_NAME	LAST_NAME	SALARY
1	Amit	Banda	6,200
2	Adam	Fripp	8,200
3	Alexander	Hunold	9,000
4	Alyssa	Hutton	8,800
5	Allan	McEwen	9,000

• MINUS

선행된 쿼리문의 수행 결과에서 후행 쿼리문의 결과값을 모두 제외함

- 아래의 쿼리는 월급 5000이상, 10000 이하이고 FIRST_NAME이 A로 시작하는 직원을 제외하는 쿼리가 됨

```
116 SELECT FIRST_NAME, LAST_NAME, SALARY
117 FROM EMPLOYEES
118 WHERE SALARY BETWEEN 5000 AND 10000
119 MINUS
120 SELECT FIRST_NAME, LAST_NAME, SALARY
121 FROM EMPLOYEES
122 WHERE FIRST_NAME LIKE 'A%';
```

Results 1

	FIRST_NAME	LAST_NAME	SALARY
9	Jose Manuel	Urman	7,800
10	Luis	Popp	6,900
11	Matthew	Weiss	8,000
12	Payam	Kaufling	7,900
13	Shanta	Vollman	6,500
14	Kevin	Mourgos	5,800
15	Peter	Tucker	10,000
16	David	Bernstein	9,500
17	Peter	Hall	9,000
18	Christopher	Olsen	8,000

• Q1. 실습

1. EMPLOYEES 테이블에서 MANAGE_ID가 108이면서 월급이 8000 이상인 직원의 데이터를 조회하되, 중복이 되는 값을 제거하고 조회 하세요
→ FIRST_NAME, LAST_NAME 컬럼을 이어 붙여서 수행 결과가 하나의 컬럼으로 나오도록 조회하세요.
2. EMPLOYEES와 DEPARTMENTS 테이블에서 DEPARTMENT_ID, MANAGER_ID가 중복 되는 데이터를 조회 하세요.

DML
(JOIN)

• JOIN

두 개 이상의 테이블의 데이터를 모두 조회할 때 사용하는 구문으로 써
관계형 데이터베이스의 경우 테이블끼리 서로 중복되는 데이터를 보관하는 경우가 많음
(* 서로 연관성을 주어 데이터에 접근하기 위해서)

이러한 관계형 데이터베이스의 특징으로 인해 JOIN을 사용하여 서로 연관된 데이터를
조회하기 위해서 많이 쓰임

• 참고

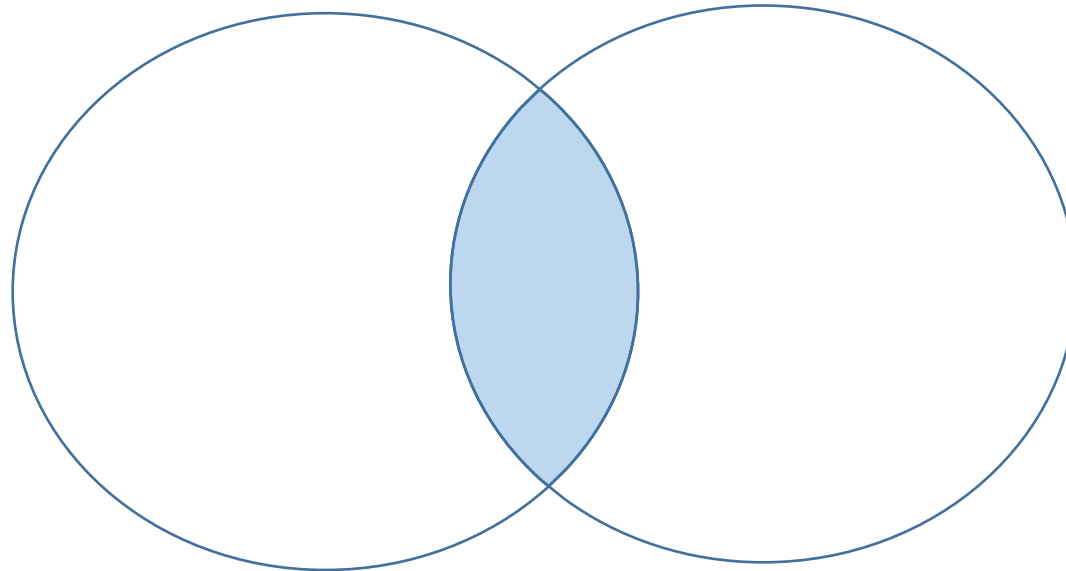
JOIN 문법은 오라클 조인과 ANSI(미국 국립 표준 협회) 조인으로 나뉘짐

• JOIN 종류

종류	설명
INNER JOIN	두 테이블의 공통된 컬럼의 값을 기준으로 집합을 반환함
LEFT OUTER JOIN	왼쪽 테이블의 모든 행과, 오른쪽 테이블에서 일치하는 행들을 반환
RIGHT OUTER JOIN	오른쪽 테이블의 모든 행과, 왼쪽 테이블에서 일치하는 행들을 반환
FULL OUTER JOIN	양쪽 테이블의 모든 행들을 반환 (오라클 구문에서는 지원하지 않고, ANSI 구문에서만 지원)
기타 등등	

- **INNER JOIN**

두 테이블의 공통된 컬럼 값을 기준으로 집합을 반환



• 오라클 조인(INNER JOIN)

EMPLOYEES 테이블과 DEPARTMENT 테이블의 데이터를 연결시켜 사용

EMPLOYEES 테이블의 DEPARTMENT_ID EMPLOYEES 테이블의 FIRST_NAME DEPARTMENT 테이블의 DEPARTMENT_NAME

```
279 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME
280 FROM EMPLOYEES e, DEPARTMENTS d
281 WHERE e.DEPARTMENT_ID = d.DEPARTMENT_ID ;
282
```

테이블 별칭

DEPARTMENT_ID가 일치하는 데이터만 합침

	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_NAME
2	20	Pat	Marketing
3	20	Michael	Marketing
4	30	Sigal	Purchasing

- ANSI 조인(INNER JOIN)

EMPLOYEES 테이블과 DEPARTMENT 테이블의 데이터를 연결시켜 사용

EMPLOYEES 테이블의
DEPARTMENT_ID

EMPLOYEES 테이블의
FIRST_NAME

DEPARTMENT 테이블의
DEPARTMENT_NAME

```
284 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME FROM EMPLOYEES e
285 JOIN DEPARTMENTS d
286 ON e.DEPARTMENT_ID = d.DEPARTMENT_ID ;
287
```

테이블 별칭

DEPARTMENT_ID가
일치하는 데이터만 합침

results 1

Results 2

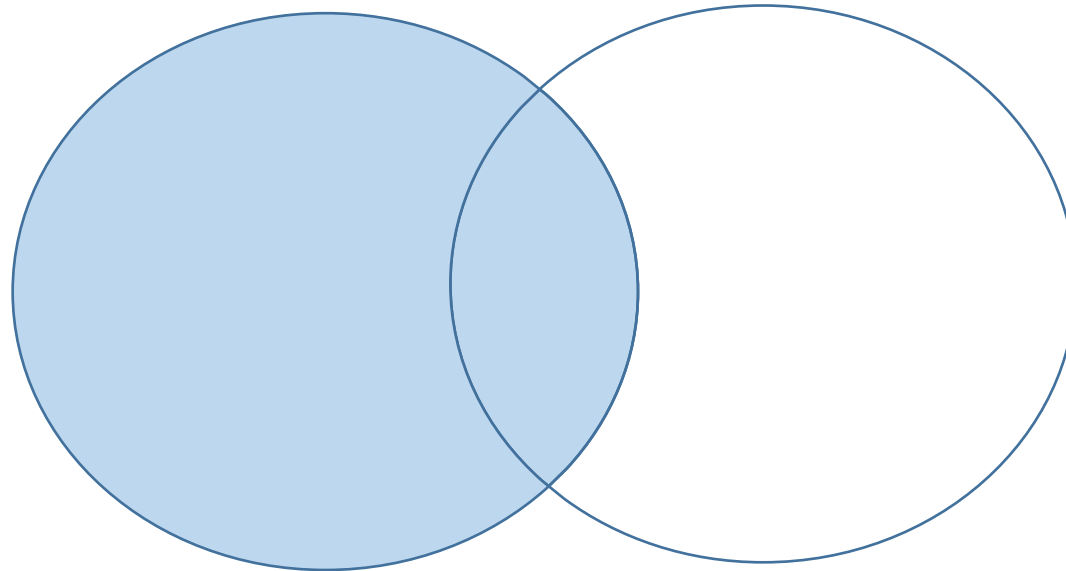
SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME

Enter a SQL expression to filter results (use Ctrl+Space)

	123	DEPARTMENT_ID	ABC	FIRST_NAME	ABC	DEPARTMENT_NAME
2		20		Pat		Marketing
3		20		Michael		Marketing

- **LEFT OUTER JOIN**

왼쪽 테이블의 모든 행과, 오른쪽 테이블에서 일치하는 행들을 반환



- 오라클 조인(LEFT OUTER JOIN)

왼쪽의 모든 행과 오른쪽의 테이블에서 일치하는 행들을 반환

```
279 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME
280 FROM EMPLOYEES e, DEPARTMENTS d
281 WHERE e.DEPARTMENT_ID = d.DEPARTMENT_ID(+);
282
```

DEPARTMENT 테이블을
EMPLOYEES 테이블로 조인 시킴

Results 1 Results 2 X

SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME

	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_NAME
1	10	Jennifer	Administration
2	20	Michael	Marketing
3	20	Pat	Marketing

- ANSI 조인(LEFT OUTER JOIN)

왼쪽의 모든 행과 오른쪽의 테이블에서 일치하는 행들을 반환

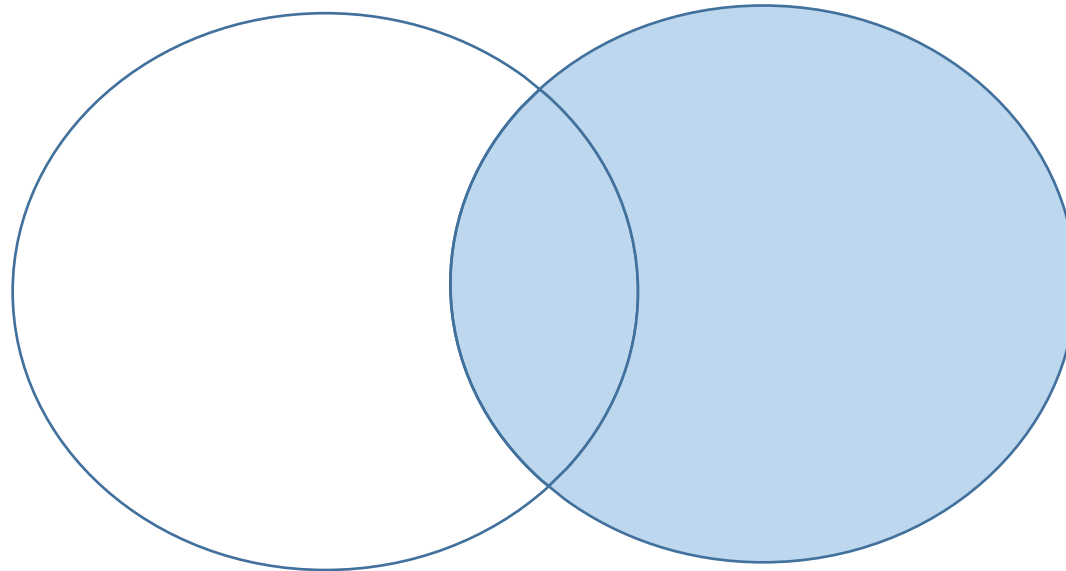
LEFT OUTER JOIN 사용 선언

```
285 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME FROM EMPLOYEES e
286 LEFT OUTER JOIN DEPARTMENTS d
287 ON e.DEPARTMENT_ID = d.DEPARTMENT_ID ;
288
```

	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_NAME
1	10	Jennifer	Administration
2	20	Pat	Marketing
3	20	Michael	Marketing
4	30	Sigal	Purchasing

- **RIGHT OUTER JOIN**

오른쪽 테이블의 모든 행과, 왼쪽쪽 테이블에서 일치하는 행들을 반환



- 오라클 조인(RIGHT OUTER JOIN)

오른쪽의 모든 행과 왼쪽의 테이블에서 일치하는 행들을 반환

EMPLOYEES 테이블을
DEPARTMENT 테이블로 조인 시킴

```
279 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME
280 FROM EMPLOYEES e, DEPARTMENTS d
281 WHERE e.DEPARTMENT_ID(+) = d.DEPARTMENT_ID ;
282
```

Results 2			
Enter a SQL expression to filter results (use Ctrl+Space)			
	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_NAME
1	10	Jennifer	Administration
2	20	Pat	Marketing
3	20	Michael	Marketing
4	30	Sigal	Purchasing

- ANSI 조인(RIGHT OUTER JOIN)

오른쪽의 모든 행과 왼쪽의 테이블에서 일치하는 행들을 반환

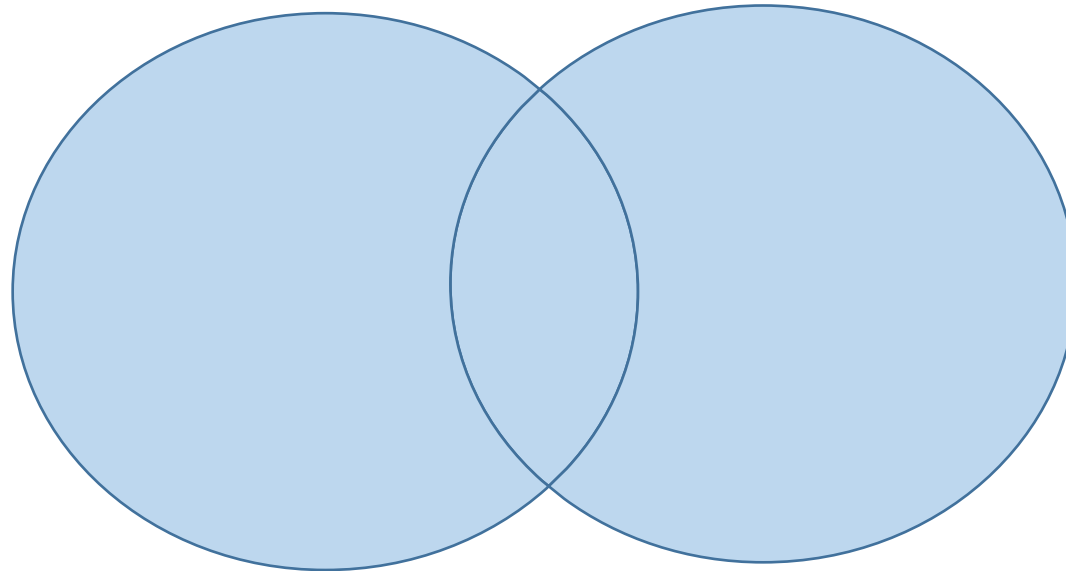
```
285 SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME FROM EMPLOYEES e
286 RIGHT OUTER JOIN DEPARTMENTS d
287 ON e.DEPARTMENT_ID = d.DEPARTMENT_ID ;
288
```

RIGHT OUTER JOIN 사용 선언

Results 1		Results 2 ×	
SELECT e.DEPARTMENT_ID, e.FIRST_NAME, d.DEPARTMENT_NAME		Enter a SQL expression to filter results (use Ctrl+Space)	
	DEPARTMENT_ID	FIRST_NAME	DEPARTMENT_NAME
1	10	Jennifer	Administration
2	20	Pat	Marketing
3	20	Michael	Marketing
4	30	Sigal	Purchasing

- **FULL OUTER JOIN**

양쪽 행들의 모든 결과를 반환



- ANSI 조인(FULL OUTER JOIN)

양쪽 행들의 모든 결과를 반환

RIGHT OUTER JOIN 사용 선언

```
SELECT * FROM EMPLOYEES e  
FULL OUTER JOIN DEPARTMENTS d  
ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
```

Results 1 X

SELECT * FROM EMPLOYEES e FULL OUTER JOIN DEPARTMENTS d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID Enter a SQL expression to filter results (use Ctrl+Space)

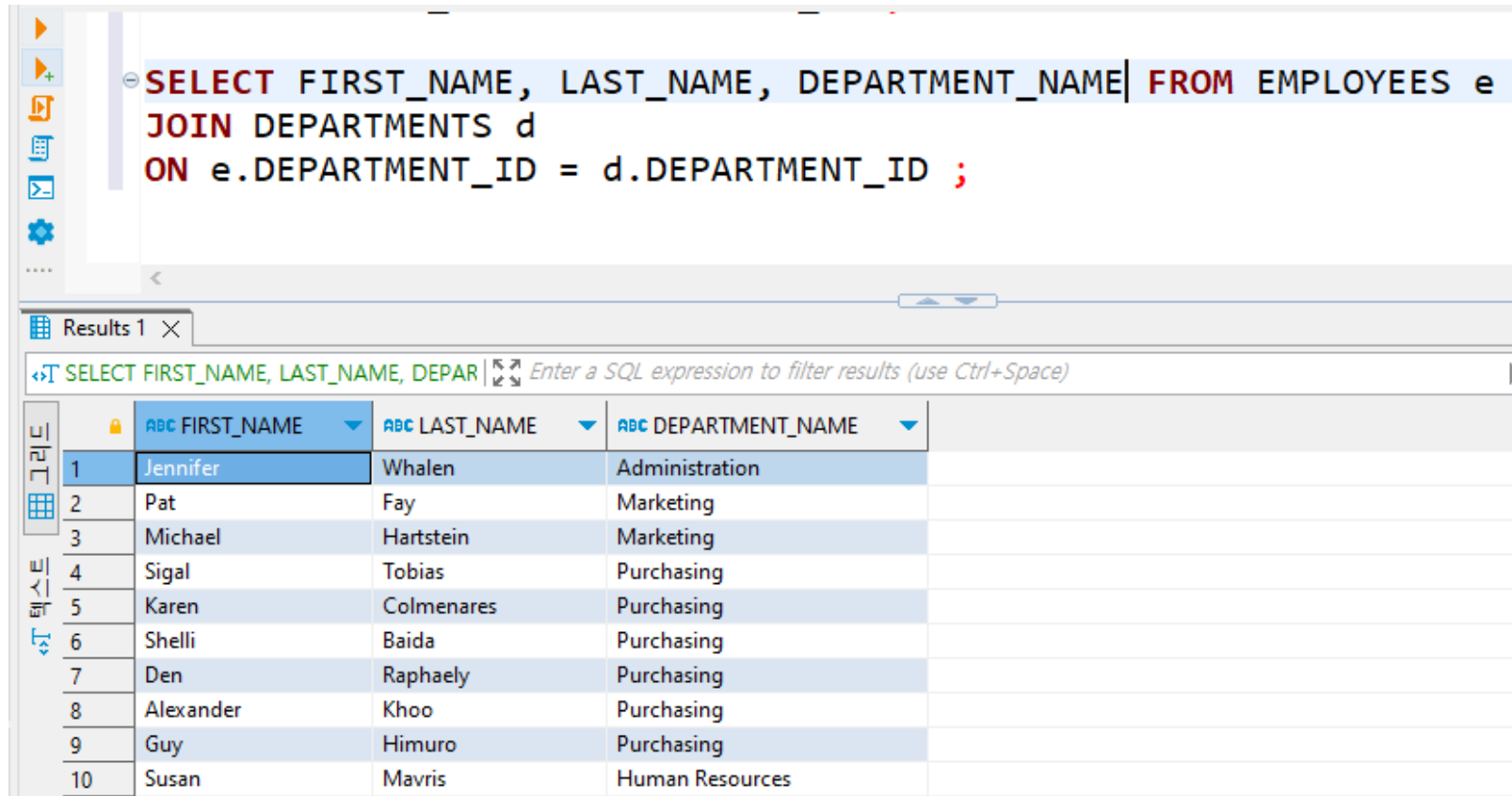
	123 EMPLOYEE_ID	ABC FIRST_NAME	ABC LAST_NAME	ABC EMAIL	ABC PHONE_N
1	100	Steven	King	SKING	515.123.4567
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568
3	102	Lex	De Haan	LDEHAAN	515.123.4569
4	103	Alexander	Hunold	AHUNOLD	590.423.4567
5	104	Bruce	Ernst	BERNST	590.423.4568
6	105	David	Austin	DAUSTIN	590.423.4569
7	106	Valli	Pataballa	VPATABAL	590.423.4560

DML

(JOIN 쿼리 작성해보기)

• JOIN 쿼리 작성해보기 - 1

INNER JOIN을 사용하여 직원들의 이름과 속한 부서의 이름 조회하기



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

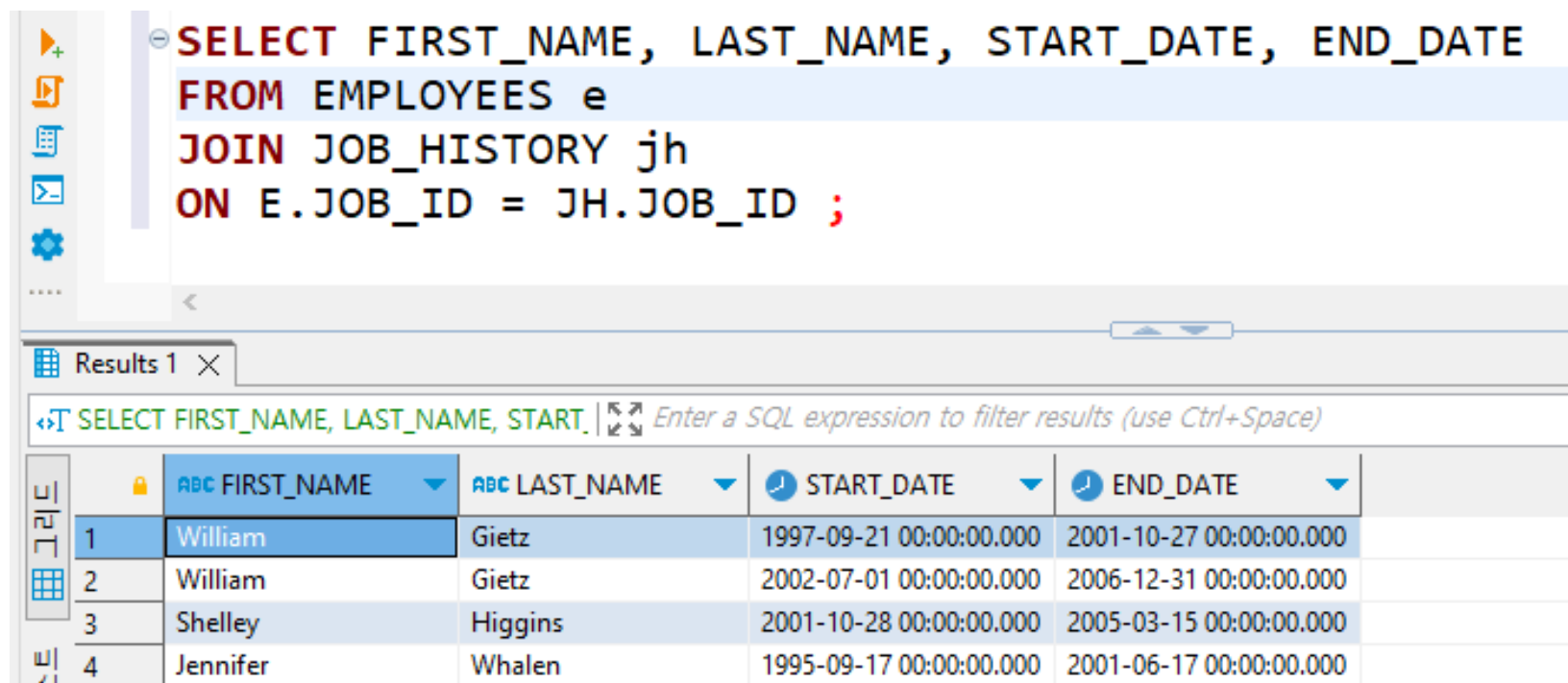
```
SELECT FIRST_NAME, LAST_NAME, DEPARTMENT_NAME FROM EMPLOYEES e  
JOIN DEPARTMENTS d  
ON e.DEPARTMENT_ID = d.DEPARTMENT_ID ;
```

The bottom pane, titled "Results 1", displays the query results in a table format. The table has three columns: FIRST_NAME, LAST_NAME, and DEPARTMENT_NAME. The results are as follows:

	FIRST_NAME	LAST_NAME	DEPARTMENT_NAME
1	Jennifer	Whalen	Administration
2	Pat	Fay	Marketing
3	Michael	Hartstein	Marketing
4	Sigal	Tobias	Purchasing
5	Karen	Colmenares	Purchasing
6	Shelli	Baida	Purchasing
7	Den	Raphaely	Purchasing
8	Alexander	Khoo	Purchasing
9	Guy	Himuro	Purchasing
10	Susan	Mavris	Human Resources

• JOIN 쿼리 작성해보기 - 2

INNER JOIN을 사용하여 직원들의 이름과 해당 직무에서 근무 기간 조회하기



The screenshot shows a SQL IDE interface. The top pane contains a SQL query: `SELECT FIRST_NAME, LAST_NAME, START_DATE, END_DATE FROM EMPLOYEES e JOIN JOB_HISTORY jh ON E.JOB_ID = JH.JOB_ID ;`. The bottom pane, titled 'Results 1', displays the query results in a table. The table has five columns: an index column, FIRST_NAME, LAST_NAME, START_DATE, and END_DATE. The results show four rows of data for the employee William Gietz, who has two job history entries, and two other employees: Shelley Higgins and Jennifer Whalen.

	ABC FIRST_NAME	ABC LAST_NAME	START_DATE	END_DATE
1	William	Gietz	1997-09-21 00:00:00.000	2001-10-27 00:00:00.000
2	William	Gietz	2002-07-01 00:00:00.000	2006-12-31 00:00:00.000
3	Shelley	Higgins	2001-10-28 00:00:00.000	2005-03-15 00:00:00.000
4	Jennifer	Whalen	1995-09-17 00:00:00.000	2001-06-17 00:00:00.000

• JOIN 쿼리 작성해보기 - 3

DEPARTMENT_ID가 100인 직원의 이름과 부서 이름 조회

```
304 SELECT E.FIRST_NAME, D.DEPARTMENT_NAME FROM EMPLOYEES e
305 JOIN DEPARTMENTS d
306 ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
307 WHERE E.DEPARTMENT_ID = 100;
308
```

Results 1

Results 2 ×

* SELECT E.FIRST_NAME, D.DEPARTMENT_NAME FROM EMPLOYEES e JOIN DEPARTMENTS d ON E.DEPARTMENT_ID = D.DEPARTMENT_ID WHERE E.DEPARTMENT_ID = 100; Enter a SQL expression to filter results (use Ctrl+Space)

	FIRST_NAME	DEPARTMENT_NAME	
1	Nancy	Finance	
2	Daniel	Finance	
3	John	Finance	

• JOIN 쿼리 작성해보기 - 4

DEPARTMENT_ID가 80이고, 급여가 12000 이상인 직원의 이름과 부서 이름 조회

```
304 SELECT E.FIRST_NAME, D.DEPARTMENT_NAME FROM EMPLOYEES e
305 JOIN DEPARTMENTS d
306 ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
307 WHERE E.DEPARTMENT_ID = 80
308 AND E.SALARY >= 12000;
```

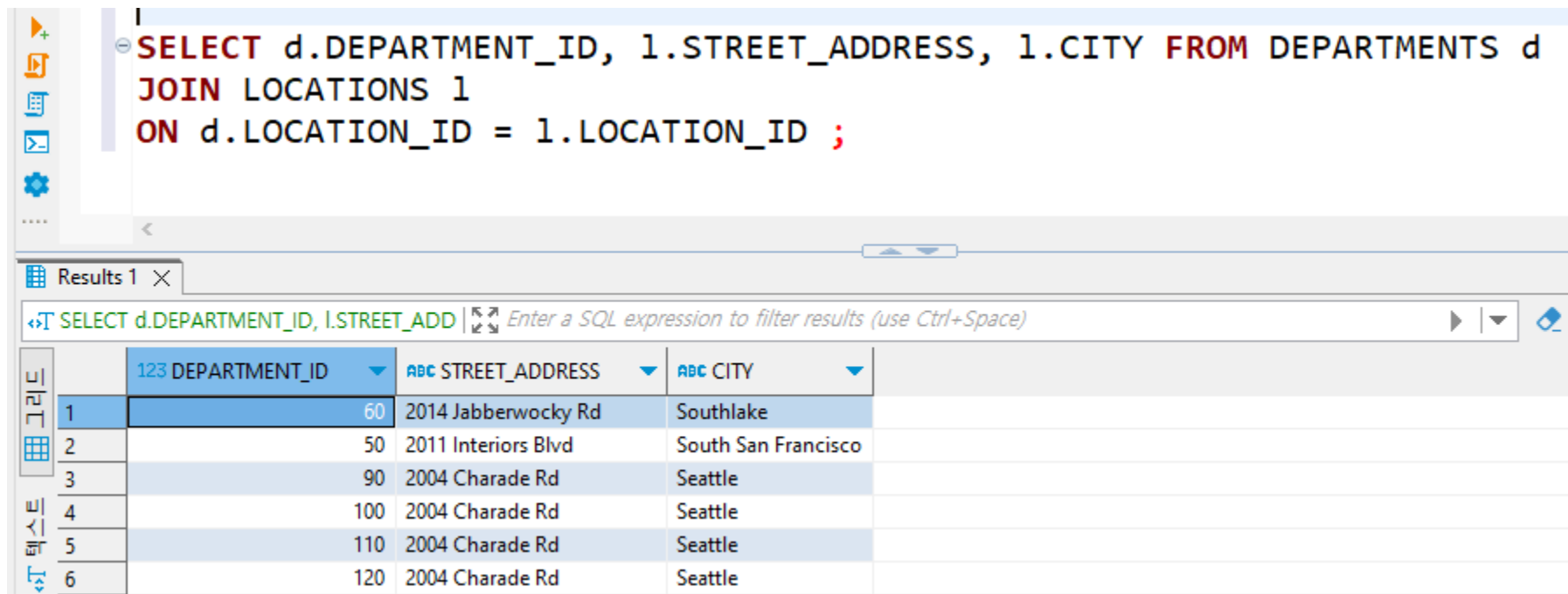
Results 1 Results 2 ×

SELECT E.FIRST_NAME, D.DEPARTMENT_NAME FROM EMPLOYEES e JOIN DEPARTMENTS d ON E.DEPARTMENT_ID = D.DEPARTMENT_ID WHERE E.DEPARTMENT_ID = 80 AND E.SALARY >= 12000;

	FIRST_NAME	DEPARTMENT_NAME
1	John	Sales
2	Karen	Sales
3	Alberto	Sales

• JOIN 쿼리 작성해보기 - 5

DEPARTMENT 테이블에서 LOCATION_ID를 기준으로 조인하여 각각의 부서 위치가 어디인지(STREET_ADDRESS, CITY) 확인하기



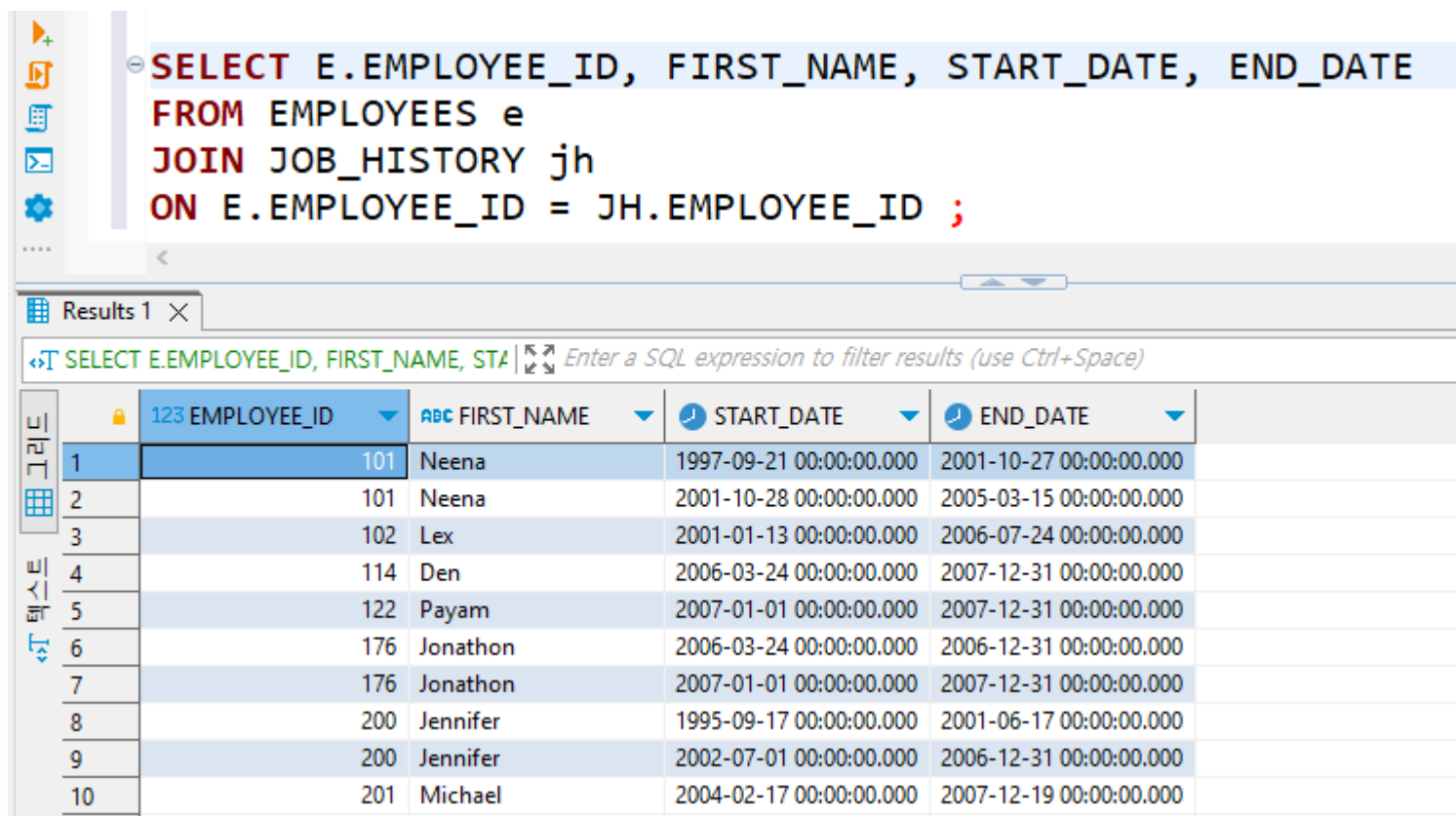
The screenshot shows a SQL IDE interface. The top pane contains a SQL query that joins the DEPARTMENTS table (aliased as d) with the LOCATIONS table (aliased as l) on the basis of their LOCATION_ID. The query selects the department ID, street address, and city. The bottom pane, titled 'Results 1', displays the output of this query as a table with 6 rows. The first row is highlighted in blue.

```
SELECT d.DEPARTMENT_ID, l.STREET_ADDRESS, l.CITY FROM DEPARTMENTS d
JOIN LOCATIONS l
ON d.LOCATION_ID = l.LOCATION_ID ;
```

	123 DEPARTMENT_ID	ABC STREET_ADDRESS	ABC CITY
1	60	2014 Jabberwocky Rd	Southlake
2	50	2011 Interiors Blvd	South San Francisco
3	90	2004 Charade Rd	Seattle
4	100	2004 Charade Rd	Seattle
5	110	2004 Charade Rd	Seattle
6	120	2004 Charade Rd	Seattle

• JOIN 쿼리 작성해보기 - 4

EMPLOYEES 테이블에서 각각의 직원이 작업한 이력 확인하기



The screenshot shows a SQL IDE interface. The top pane contains a SQL query that joins the EMPLOYEES and JOB_HISTORY tables. The bottom pane shows the results of this query, which is a table with 10 rows and 5 columns: EMPLOYEE_ID, FIRST_NAME, START_DATE, and END_DATE. The first row is highlighted in blue.

```
SELECT E.EMPLOYEE_ID, FIRST_NAME, START_DATE, END_DATE
FROM EMPLOYEES e
JOIN JOB_HISTORY jh
ON E.EMPLOYEE_ID = JH.EMPLOYEE_ID ;
```

	EMPLOYEE_ID	FIRST_NAME	START_DATE	END_DATE
1	101	Neena	1997-09-21 00:00:00.000	2001-10-27 00:00:00.000
2	101	Neena	2001-10-28 00:00:00.000	2005-03-15 00:00:00.000
3	102	Lex	2001-01-13 00:00:00.000	2006-07-24 00:00:00.000
4	114	Den	2006-03-24 00:00:00.000	2007-12-31 00:00:00.000
5	122	Payam	2007-01-01 00:00:00.000	2007-12-31 00:00:00.000
6	176	Jonathon	2006-03-24 00:00:00.000	2006-12-31 00:00:00.000
7	176	Jonathon	2007-01-01 00:00:00.000	2007-12-31 00:00:00.000
8	200	Jennifer	1995-09-17 00:00:00.000	2001-06-17 00:00:00.000
9	200	Jennifer	2002-07-01 00:00:00.000	2006-12-31 00:00:00.000
10	201	Michael	2004-02-17 00:00:00.000	2007-12-19 00:00:00.000

- **EMPLOYEE_ID에만 별칭으로 테이블을 정의한 이유**

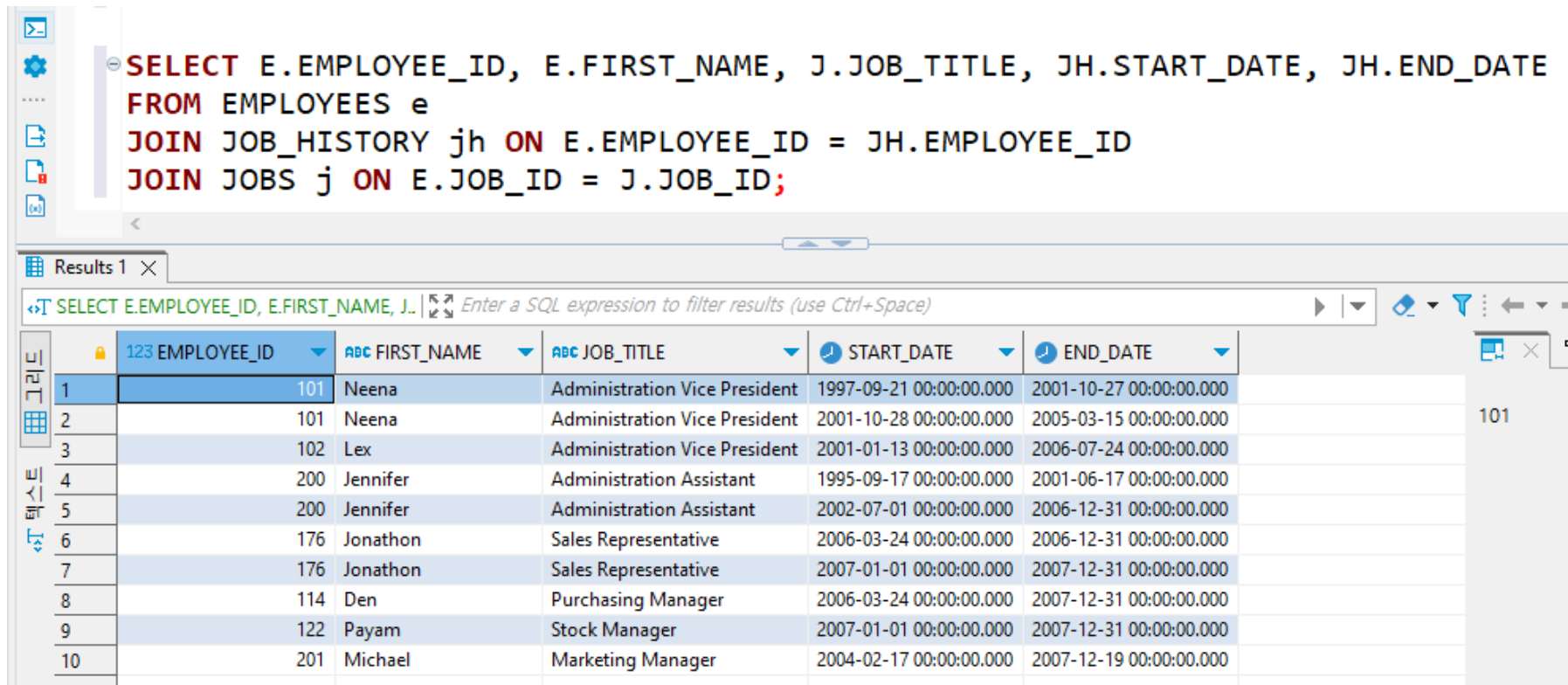
EMPLOYEES 테이블과 JOB_HISTORY 테이블 모두 EMPLOYEE_ID를 가지고 있기 때문에 어떤 테이블의 EMPLOYEE_ID 컬럼을 가지고 올건지 명확하게 작성해 주어야 함.

그 외 다른 컬럼(FIRST_NAME, START_DATE, END_DATE)은 각각의 테이블에 따로 존재하기 때문에 별칭을 지어주지 않아도 사용 가능함

• JOIN 쿼리 작성해보기 - 5

EMPLOYEES 테이블에서 각각의 직원이 작업한 이력, 직무명 확인하기

* 여러 개의 테이블을 조인할 수 있음



The screenshot shows a SQL IDE interface. The top pane contains a SQL query that joins the EMPLOYEES, JOB_HISTORY, and JOBS tables. The bottom pane shows the results of this query in a table format. The table has columns for EMPLOYEE_ID, FIRST_NAME, JOB_TITLE, START_DATE, and END_DATE. The results list 10 rows of employee job history.

```
SELECT E.EMPLOYEE_ID, E.FIRST_NAME, J.JOB_TITLE, JH.START_DATE, JH.END_DATE
FROM EMPLOYEES e
JOIN JOB_HISTORY jh ON E.EMPLOYEE_ID = JH.EMPLOYEE_ID
JOIN JOBS j ON E.JOB_ID = J.JOB_ID;
```

	EMPLOYEE_ID	FIRST_NAME	JOB_TITLE	START_DATE	END_DATE
1	101	Neena	Administration Vice President	1997-09-21 00:00:00.000	2001-10-27 00:00:00.000
2	101	Neena	Administration Vice President	2001-10-28 00:00:00.000	2005-03-15 00:00:00.000
3	102	Lex	Administration Vice President	2001-01-13 00:00:00.000	2006-07-24 00:00:00.000
4	200	Jennifer	Administration Assistant	1995-09-17 00:00:00.000	2001-06-17 00:00:00.000
5	200	Jennifer	Administration Assistant	2002-07-01 00:00:00.000	2006-12-31 00:00:00.000
6	176	Jonathon	Sales Representative	2006-03-24 00:00:00.000	2006-12-31 00:00:00.000
7	176	Jonathon	Sales Representative	2007-01-01 00:00:00.000	2007-12-31 00:00:00.000
8	114	Den	Purchasing Manager	2006-03-24 00:00:00.000	2007-12-31 00:00:00.000
9	122	Payam	Stock Manager	2007-01-01 00:00:00.000	2007-12-31 00:00:00.000
10	201	Michael	Marketing Manager	2004-02-17 00:00:00.000	2007-12-19 00:00:00.000

• Q1. INNER JOIN 실습

1. 직원의 이름, 이메일, 직무이름을 조회하세요.
→ EMPLOYEES, JOBS 테이블 조인 사용
2. 직원의 이름, 속한 부서 이름, 직무, 근무 위치를 조회하세요.
→ EMPLOYEES, DEPARTMENT, JOBS, LOCATIONS

• Q2. INNER JOIN + 조건식 실습

1. 부서 이름이 'Sales' 이거나 'Marketing' 이고, 급여가 5000 이상인 직원 이름 조회
→ EMPLOYEES, DEPARTMENT 테이블
→ 부서 이름 : DEPARTMENT_TITLE
→ 급여 : SALARY
2. 입사일이 2005년 6월 30일 이전인 직원의 직무명 조회
→ EMPLOYEES, JOBS 테이블
→ 입사일 : HIRE_DATE
→ 직무명 : JOB_TITLE
3. 부서 이름이 'Shipping' 이고, 이름이 7글자 이상인 직원의 아이디 조회
→ EMPLOYEES, DEPARTMENT 테이블
→ 부서 이름 : DEPARTMENT_NAME
→ 이름 : FIRST_NAME
→ 직원 아이디 : EMPLOYEE_ID

• JOIN + GROUPBY 쿼리 작성해보기 - 1

각 부서별로 직원 수 검색하여 결과 출력

```
SELECT D.DEPARTMENT_NAME, COUNT(E.EMPLOYEE_ID) FROM EMPLOYEES e  
JOIN DEPARTMENTS d  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID  
GROUP BY DEPARTMENT_NAME;
```

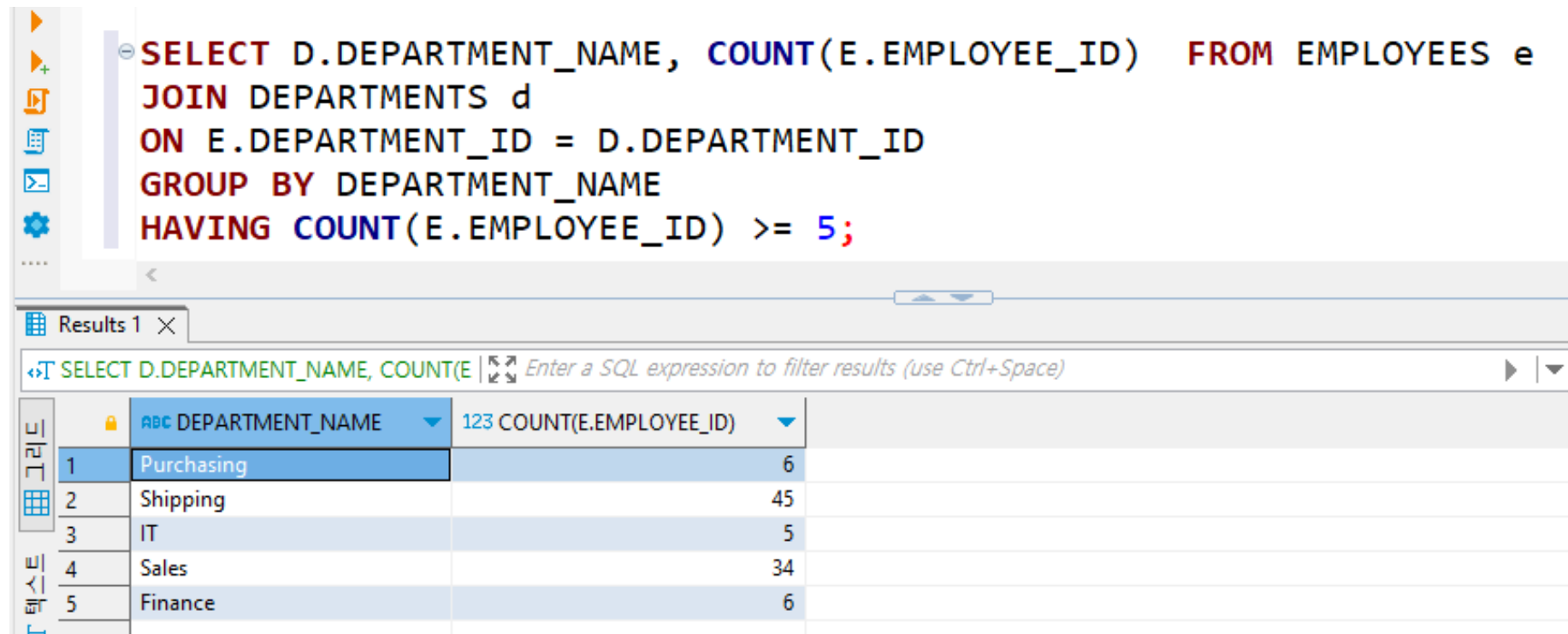
Results 1 X

SELECT D.DEPARTMENT_NAME, COUNT(E.EMPLOYEE_ID) Enter a SQL expression to filter results (use Ctrl+Space)

	ABC DEPARTMENT_NAME	123 COUNT(E.EMPLOYEE_ID)
1	Administration	1
2	Marketing	2
3	Purchasing	6
4	Human Resources	1
5	Shipping	45
6	IT	5
7	Public Relations	1
8	Sales	34
9	Executive	3
10	Finance	6
11	Accounting	2

• JOIN + GROUPBY 쿼리 작성해보기 - 2

각 부서별로 직원 수 검색하여 결과 출력하되 직원 수가 5명 이상인 부서



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

```
SELECT D.DEPARTMENT_NAME, COUNT(E.EMPLOYEE_ID) FROM EMPLOYEES e  
JOIN DEPARTMENTS d  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID  
GROUP BY DEPARTMENT_NAME  
HAVING COUNT(E.EMPLOYEE_ID) >= 5;
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has two columns: "DEPARTMENT_NAME" and "COUNT(E.EMPLOYEE_ID)". The results are as follows:

	DEPARTMENT_NAME	COUNT(E.EMPLOYEE_ID)
1	Purchasing	6
2	Shipping	45
3	IT	5
4	Sales	34
5	Finance	6

• JOIN + GROUPBY 쿼리 작성해보기 - 2

각 직무별로 평균 급여를 검색하고, 평균 급여가 가장 높은 직무의 이름과 평균 급여 출력하기

```
SELECT J.JOB_TITLE , AVG(e.salary) AS AVG_SALARY  
FROM EMPLOYEES e  
INNER JOIN JOBS j ON E.JOB_ID = J.JOB_ID  
GROUP BY J.JOB_TITLE  
ORDER BY AVG_SALARY DESC  
FETCH FIRST 1 ROW ONLY;
```

Results 1	
SELECT J.JOB_TITLE , AVG(e.salary) AS AVG_SALARY	
ABC JOB_TITLE	123 AVG_SALARY
President	24,000